Experiment No. 13 : Shortest path

```
#include<stdio.h>
#include<stdlib.h>
#define MAX_NODES 10
#define INFINITY 10000
//ENUMERATION FOR MAKING NODE PERMANENT
enum node
{
permanent,tentative
};
int path[MAX_NODES],dist[MAX_NODES][MAX_NODES];
int src,dst,n,count=0;
//NODE STRUCTURE
struct state
{
int predecessor;
int length;
enum node lbl;
}state[MAX_NODES];
void shortestpath(int ,int );
void main()
{
int i,j ;
printf("\n\n\t\tSHORTEST PATH CALCULATION \n");
printf("\nEnter the number of nodes(LESS THAN 10): ");
scanf("%d",&n);
for(i=0;i<n;i++)
for(j=0;j<n;j++)
dist[i][j]=0;
for(i=0;i<n;i++)
{
for(j=0;j<n;j++)
{
if((dist[i][j]==0) && (i!=j))
{
```

```
printf("Enter the distance between %d & %d : ",i+1,j+1);
scanf("%d",&dist[i][j]);
dist[j][i]=dist[i][j];
}
}
}
printf("\nDistance Matrix\n");
for(i=0;i<n;i++)
{
for(j=0;j<n;j++)
printf("\t%d",dist[i][j]);
printf("\n");
}
printf("\nEnter source node: ");
scanf("%d",&src);
printf("\nEnter destination node:");
scanf("%d",&dst);
if(src==dst)
{
printf("\nThere is no path for self\n");
exit(0);
}
shortestpath(src-1,dst-1);
printf("\nThe shortest path is:");
while(count>0)
{
if(count==1)
printf("%d",path[count-1]+1);
else
printf("%d==>",path[count-1]+1);
count--;
}
printf("\n\nDistance of shortest path is: %d\n",state[dst-1].length);
}
void shortestpath(int s,int d)
```

```
{
int i,k,min,j;
struct state *p;
for(p=&state[0];p<&state[n];p++)
{
p->predecessor=-1;
p->length=INFINITY;
p->lbl=tentative;
}
//MAKE SOURCE NODE PERMANENT
state[s].length=0;
state[s].lbl=permanent;
k=s;
do
{
for(i=0;i<n;i++)
{
if(dist[k][i]!=0 && state[i].lbl==tentative)
{
if((state[k].length+dist[k][i]) < state[i].length)
{
state[i].predecessor=k;
state[i].length=state[k].length+dist[k][i];
}
}
}
k=0;
min=INFINITY;
for(i=0;i<n;i++)
{
if(state[i].lbl==tentative && (state[i].length< min))
{
min=state[i].length;
k=i;
}
```

```
}
state[k].lbl=permanent;
}while(k!=d);
i=0;
k=d;
do
{
path[i++]=k;
k=state[k].predecessor;
count++;
}while(k>=0);
}
```

Output :

